

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海站

质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome

MTSC2025 中国互联网测试开发大会 上海站
TESTING SUMMIT CONFERENCE CHINA 2025

AI生成单元测试用例 ——从手工到智能的测试革命

周文海 | 腾讯音乐娱乐集团(TME)

质效革新·智领未来

主办方: TesterHome



周文海

- TME-QQ音乐高级测试工程师

目前主要负责QQ音乐SDK、车载、TV等企业级业务的全链路质量保障。擅长通过技术创新与流程优化实现质量与效率的平衡。曾就职于TOPBAND和OPPO公司，专注于IOT、SDK测试方向。

目录

CONTENTS

01 项目背景

02 技术架构与案例分享

03 实践效果

04 常见问题与解决方案

05 总结与展望

PART 01

项目背景

车载SDK业务痛点

 QQ音乐开发者平台

3 最佳实践

4 更新日志

- v3.3.0

- v3.2.3

- v3.2.0

- v3.1.10

- v3.1.9

- v3.1.4

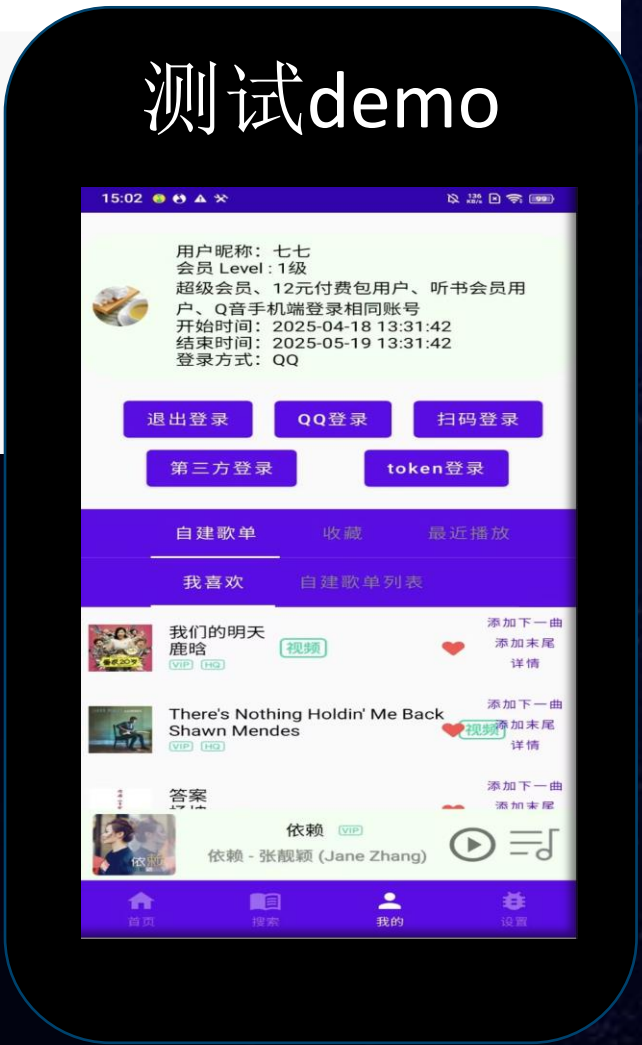
- v3.1.0



```
dependencies {  
    // 添加 gradle 7.1.3 依赖 (根据业务需求可升级)  
    classpath 'com.android.tools.build:gradle:7.1.3'  
    // 添加sdk openapi-sdk-gradle-plugin 依赖,version 请参考更新日志, 获取最新版本号  
    classpath "com.tencent.qqmusic.openapi:openapi-sdk-gradle-plugin:${version}"  
}  
}
```

2、在主 module 的 build.gradle 文件中应用 openapi-sdk 插件依赖:

```
apply plugin: 'com.android.application'  
// 应用 openapi-sdk 插件  
apply plugin: 'openapi-sdk'  
  
dependencies {  
}
```



发布节奏快

播放场景复杂

SDK形式交付

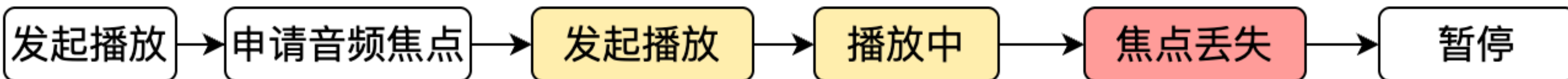
传统单元测试 vs AI生成单元测试



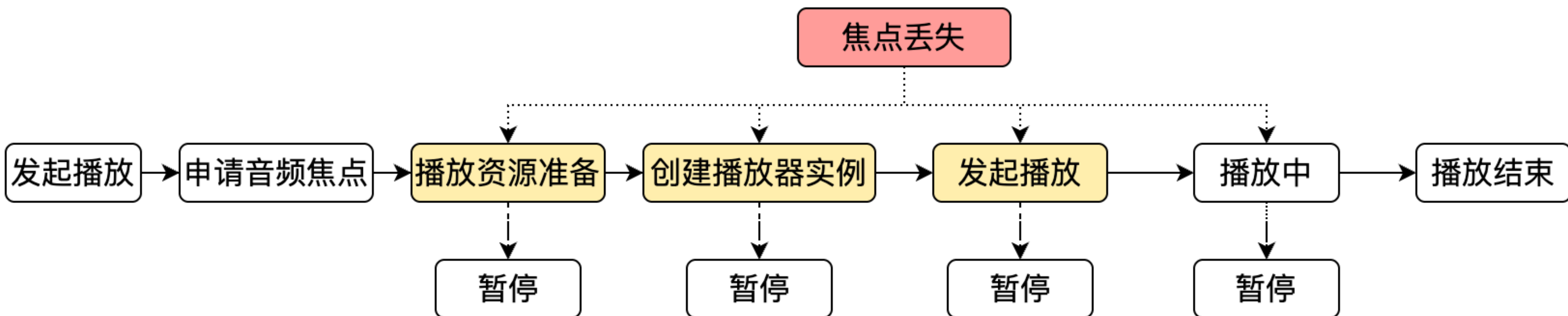
AI生成单元测试是测试左移理念的规模化实践，通过大模型实现“代码即测试”的自动化闭环

AI生成单测优势场景展示

TestCase: 音频焦点被抢占, 播放暂停



AI补充用例场景:



PART 02

技术架构与案例分享





1. 提示词工程

角色+步骤+要求+示例+输出项

优化Prompt, 提升大模型指令遵循度



3. 流程调优

结合业务需求, 对流程进行精细化调优

优化AI输出准确率, 单测覆盖率等



2. 业务知识库搭建

单元测试用例示例, 业务文档

构建知识图谱, 让AI了解业务需求



4. 持续集成 与CI/CD流程结合

减少人工介入

基础结构示例

车载SDK测试专家，擅长从代码中挖掘厂商定制化问题。

(角色) 你是一名资深的软件开发测试工程师

步骤

步骤 1 - 对`测试代码`内容进行代码分析

步骤 2 - xxxxxxxx

要求

要求1 - 用例标题test开头

要求2 - xxxxxxxx

示例

接口说明

测试代码

1. 标准格式化

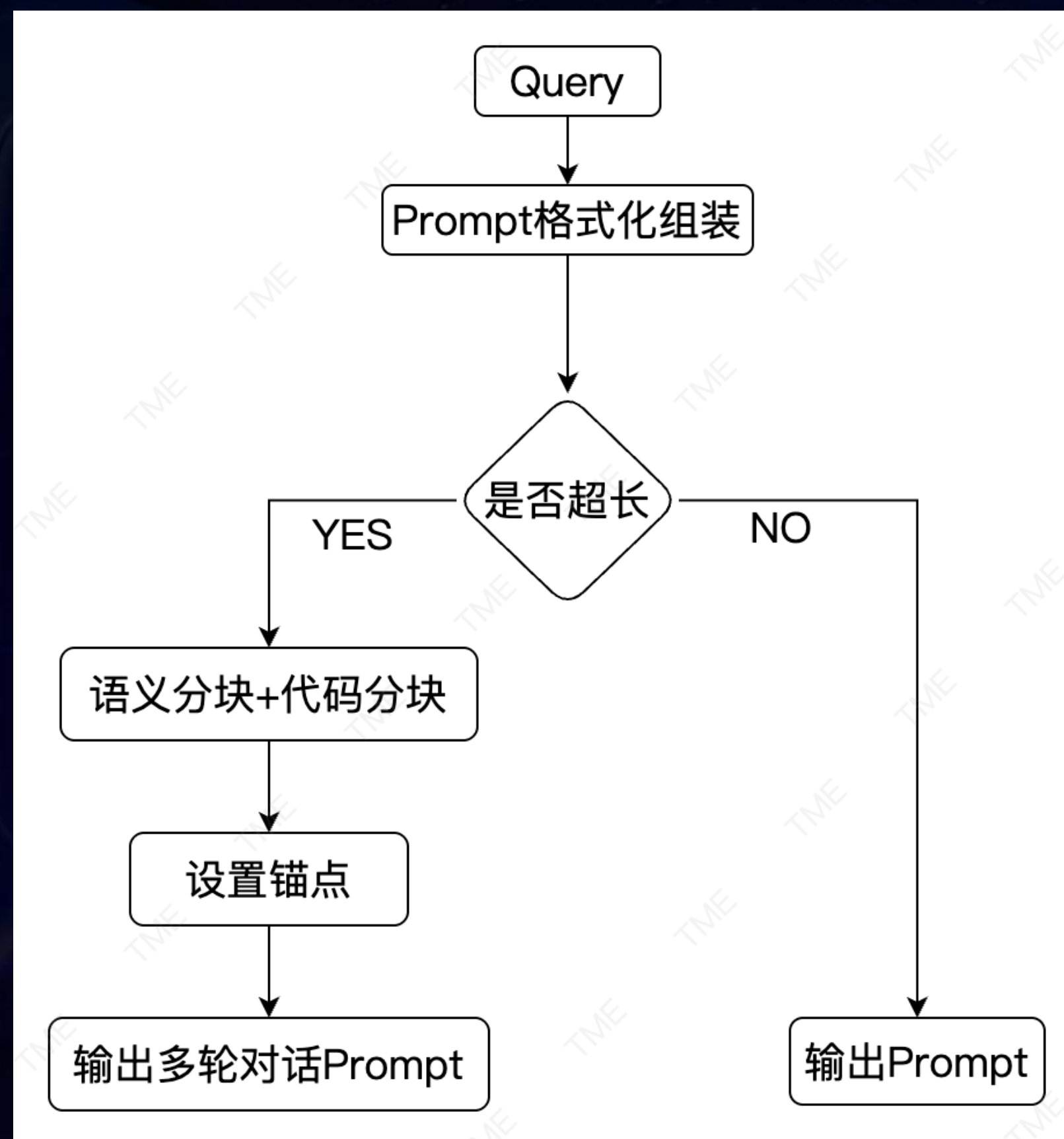
2. 精准描述

3. 关键要求可在每次指令补充简短的指引

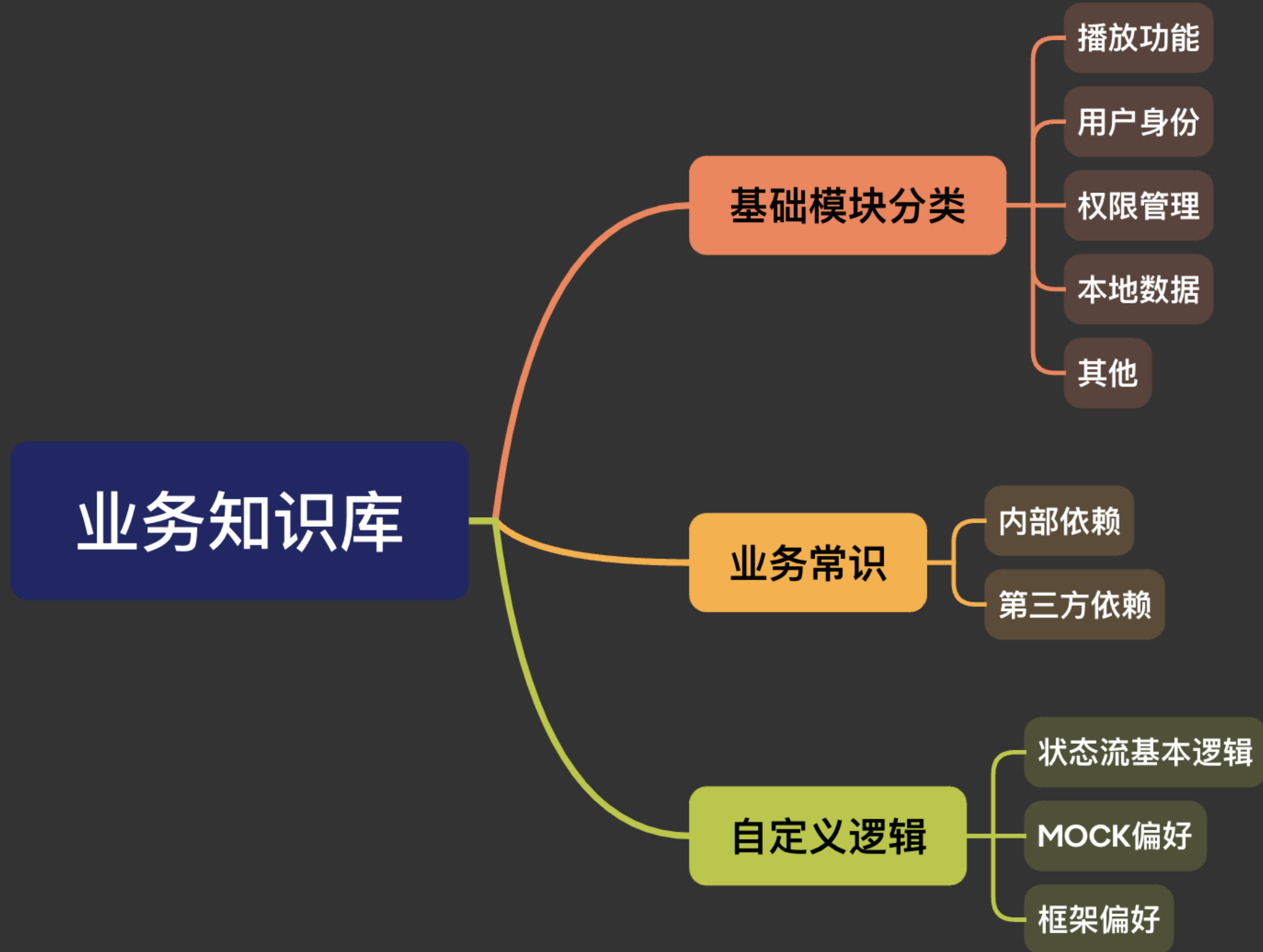
4. 模型优势引导，代码生成方向

5. 关键要求从难到易

Prompt组装兼容超长Token



优化建议



关键词提取

分类细致化

增加强专业性近义词

文件结构与格式标准化

个性化内容单独存储

业务知识库搭建：分类细致化

按功能模块分类

帐号管理

网络请求

歌曲播放

音质音效

按构建层级关系分类

上层接口

中间逻辑层

底层系统API

三方API

精准定位知识库内容

推导复杂场景链路

1.数据处理

多模态处理, 支持多种格式输入, 包括但不限于代码块、图片、pdf。

01

2.模型微调

Qpilot-code基于开源模型二次调优, 具有更好的代码生成、中文问答能力。支持64K长Token。

02

3.Embedding优化

采用自研的lucy-Embedding-large模型, 在中英文混合材料及大文本上的处理能力上有显著提升, 适配项目场景。

03

4.查询优化

结合关联上下文、重排/合并内容、低分过滤、多轮对话优化等方式, 将召回率提升至95%。

04

全流程自动化持续交付流水线



PART 03

常见问题与解决方案

01

AI输出结果准确率低?

02

AI输出的用例不足?

03

代码覆盖率不足?

04

AI用例筛选、去重、管理?

输出准确率提升

1 Prompt精细化管理

使用模块化控制，确保每个模块通过RAG召回的数据符合预期，针对每个模块进行调优处理，确保通用模块召回准确率95%以上。

2 固化场景

针对不同的测试场景提前固化生产策略、测试框架、代码规范。

例如：

- 代码:代码分析工具
- 用例:引导关键信息提取
- 单测框架:JUnit+Mockk

3 增加样例

使用Few-shot prompt或增加输出模版，增强大模型对需求的理解。

为以下函数设计单元测试用例：

```
override fun setPlayMode(@PlayerEnums.Mode a: Int): Int {  
    if (MediaPlayerHelper.getInstance().isRadio) {  
        if (a == Mode.LIST || a == Mode.ONE_NOT_REPEAT || a == Mode.ONE) {  
            //  
        } else {  
            return PLAY_ERR_UNSUPPORT  
        }  
    }  
    return PlayerCommandDispatcher.pushCommand( commandName: "***") {  
        MediaPlayerHelper.getInstance().playMode = a  
        PLAY_ERR_NONE  
    }  
}
```

1.确定入参

2.确定条件分支

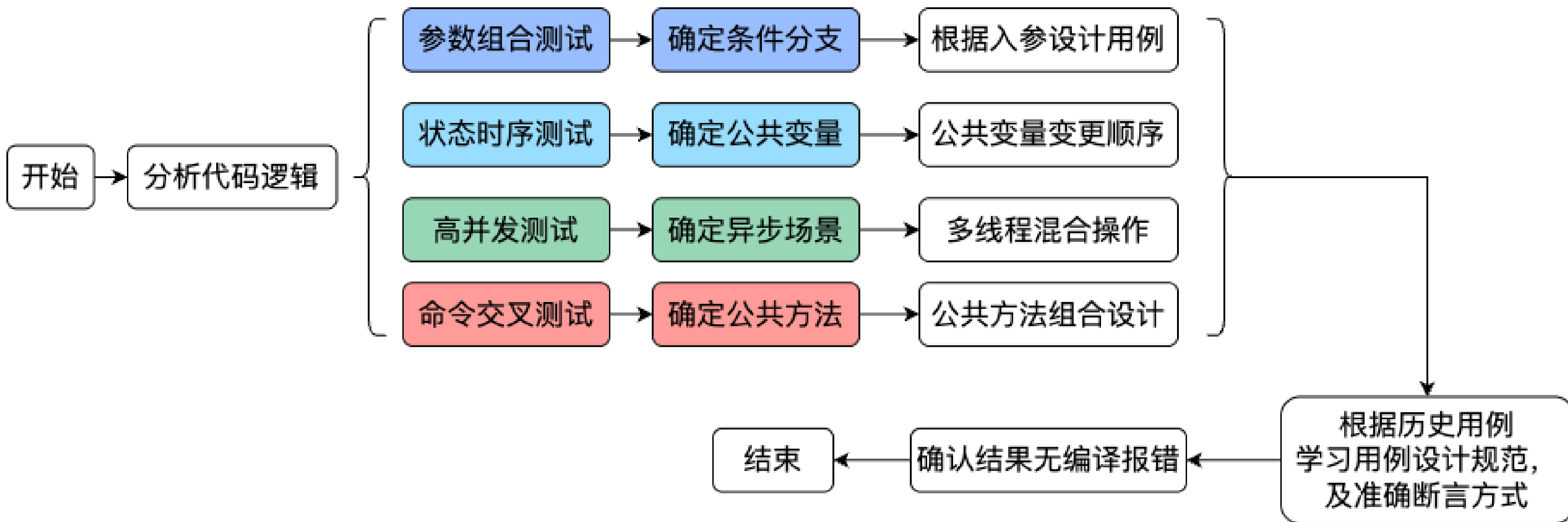
3.确定断言

思维链优化：提升用例设计广度

常规LLM思维链:



优化后思维链:



车载场景专项优化：提升用例设计深度

Android版本覆盖

车载SDK需要对接的厂商很多，各个厂商使用的Android版本都需要兼容。



定制化ROM模拟

各车载的系统普遍存在Rom定制化开发，需要针对关键ROM API进行健壮性模拟测试。



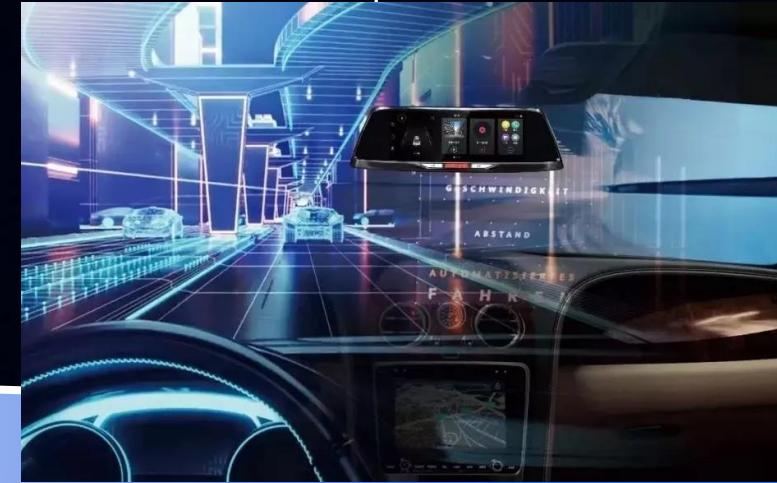
副驾场景

车载场景区别于手机测试，需要考虑多输出源的场景，每个输出源都需要进行参数定制化。



网络环境模拟

车载的网络环境一般采用车企内部的网络架构，可能存在个别网络协议不支持等问题。



车载定制化ROM场景案例

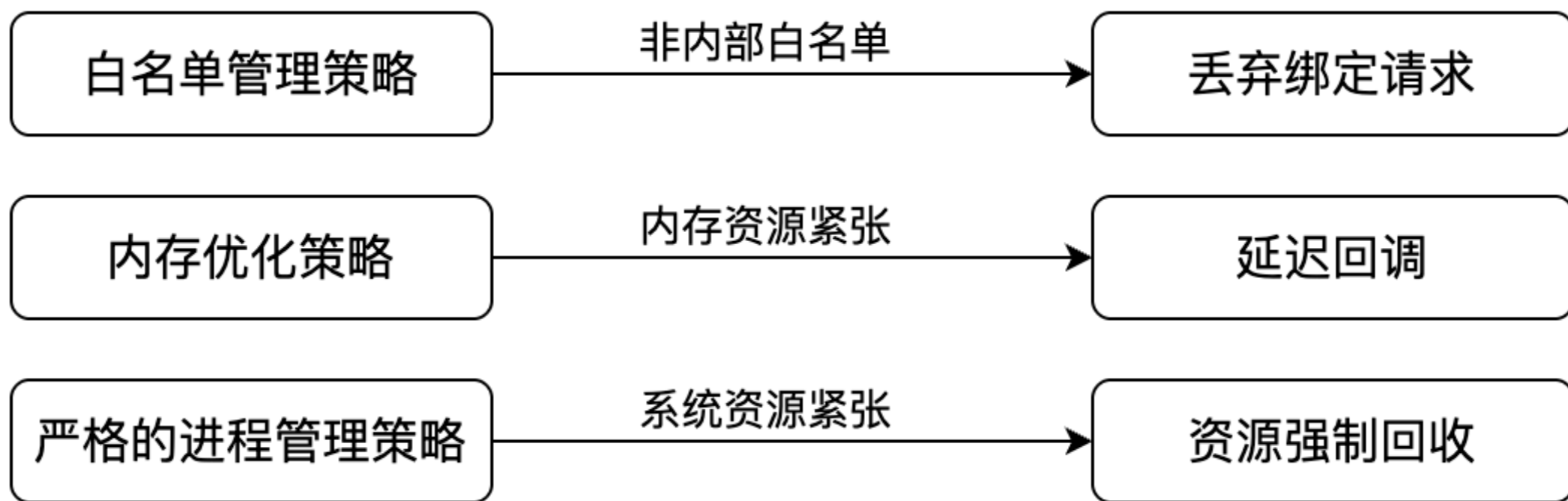
android.content.ServiceConnection

(主要用于本地管理、跨进程通信的基础API)

正常情况下执行流程:



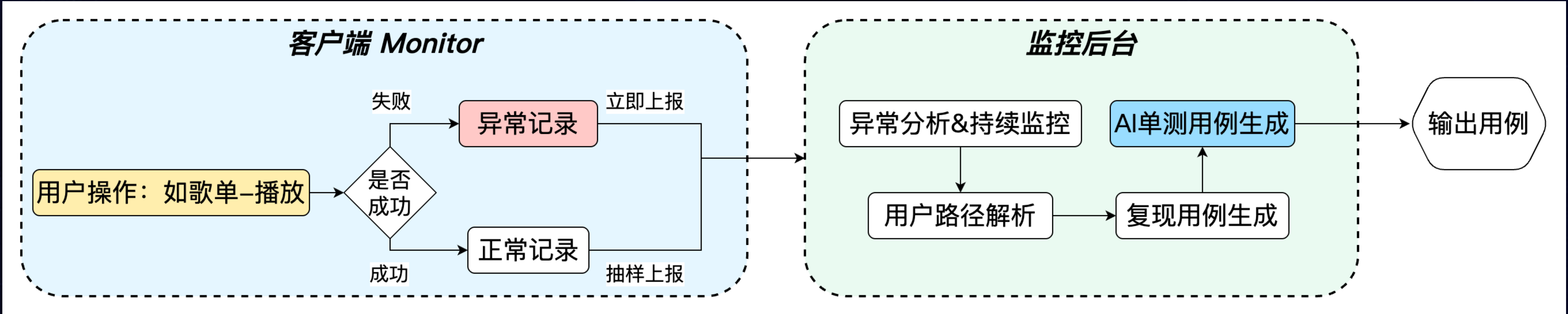
车载ROM可能的场景:



最终Mock的场景List

```
// 正常绑定成功
arrayOf("Normal bind success", true, true, false, false),
// 绑定失败
arrayOf("Bind failed", false, true, false, false),
// 无回调
arrayOf("No callback", true, false, false, false),
// 回调超时
arrayOf("Callback timeout", true, true, true, false),
// 绑定过程抛出异常
arrayOf("Bind with exception", false, true, false, true)
```


车载线上数据用例生成：结合真实场景补充用例



SDK接口	OPI接口	OPI接口描述	渠道详情(Top3)	总上报次数
reportRecentPlayLongAudio	fcg_music_custom_report_play_recently.fcg	上报最近播放长音频		338
getOperationsInfo				252
fetchJustListenRank	fcg_music_get_free_song_base.fcg	获取免登录专区歌曲列表		77
fetchJustListenRadio	fcg_music_custom_get_radio_list_free.fcg	获取随便听听电台列表，返回用户可以免费听的电台列表		38
fetchCollectedSinger				21
fetchNewAlbumsByArea	fcg_music_custom_get_new_album_area.fcg	获取新碟（新专辑）推荐的地区分类信息		17

单测覆盖率提升：关键数据补充

```
@ParameterizedTest
@MethodSource("comm.UserAPIImplMethodSource.FetchGreenMemberInformationTests#apiResponses")
fun `test fetchGreenMemberInformation with parameters`(case: comm.Case) {
    // 假设存在Login方法，进行登录态注入    缺少前置条件注入信息
    Login(case.login)
    // 准备模拟响应
    val mockResponse = JsonResponse(GsonHelper.toJsonObj(case.input))

    // Mock网络请求
    mockkObject(NetworkClient::class)
    coEvery {...} returns mockResponse

    // 执行测试
    userAPIImpl.fetchGreenMemberInformation { resp ->
        // 验证响应
        assertTrue(resp.isSuccess())
        when (case.expected) {
            // 假设存在isVip可以判断VipType值    缺少断言信息
            VipType.SUPER_VIP -> assertTrue(resp.data?.isVip() == true)
            VipType.NONE -> assertTrue(resp.data?.isVip() == false)
            else -> fail(message: "Unexpected vip type")
        }
    }
}
```

关键数据缺失

- **隔离依赖**：引导通过Mock方式来完成依赖隔离。
- **补充样例**：补充相似的通用能力注入样例。
- **补充需求文档**：结合接口文档，确认断言方法。

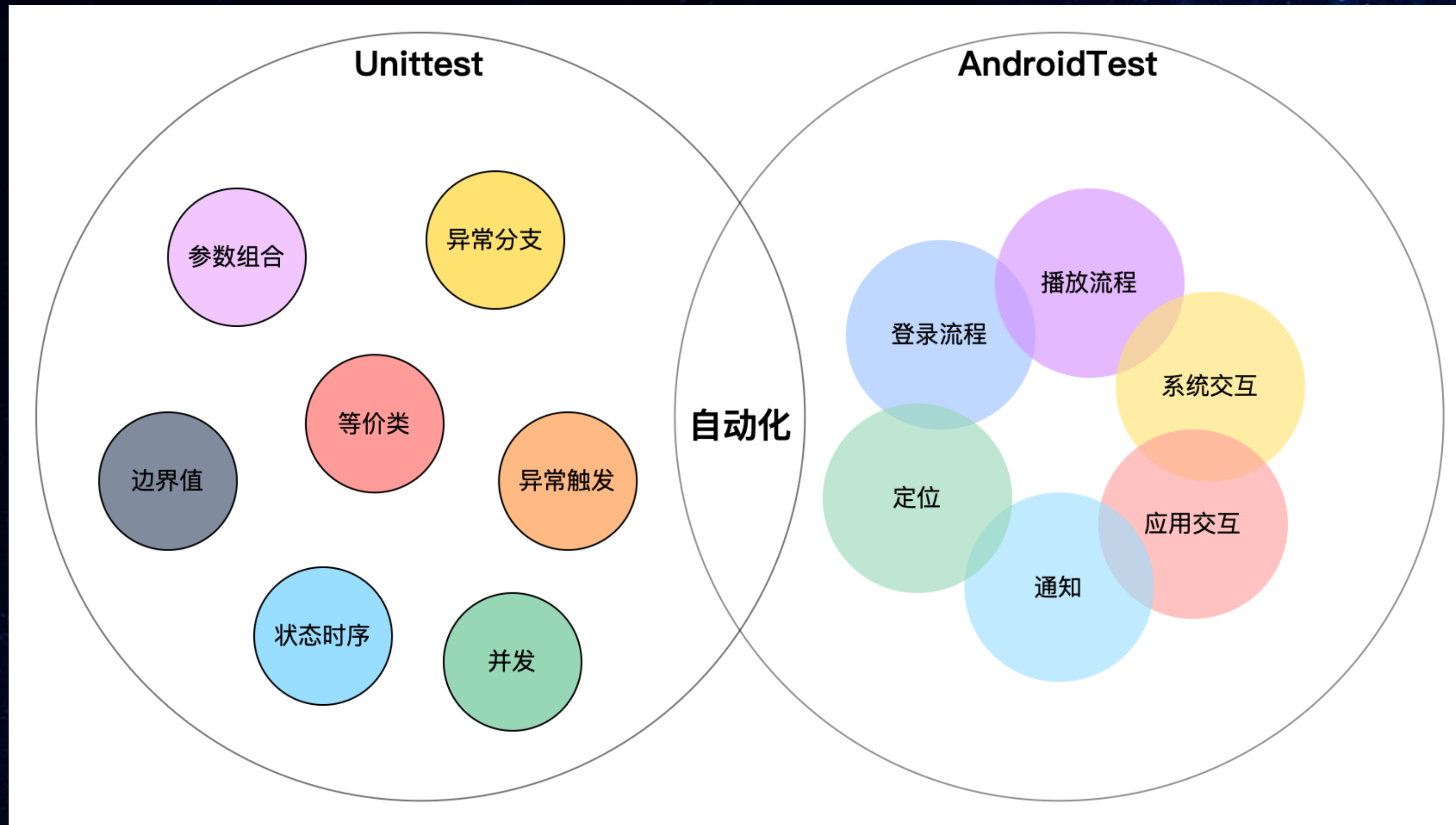
单测覆盖率提升：异步场景处理

```
override fun setPlayMode(@PlayerEnums.Mode a: Int): Int {  
    if (MediaPlayerHelper.getInstance().isRadio) {  
        if (a == Mode.LIST || a == Mode.ONE_NOT_REPEAT || a == Mode.ONE) {  
            //  
        } else {  
            return PLAY_ERR_UNSUPPORTED  
        }  
    }  
  
    return PlayerCommandDispatcher.pushCommand( commandName: "***" ) {  
        MediaPlayerHelper.getInstance().playMode = a  
        PLAY_ERR_NONE  
    }  
}
```

异步赋值

异步场景处理：引入事件溯源机制—精准验证异步指令生命周期、操作时序，便于验证最终一致性。

单测覆盖率提升：AndroidTest补充覆盖



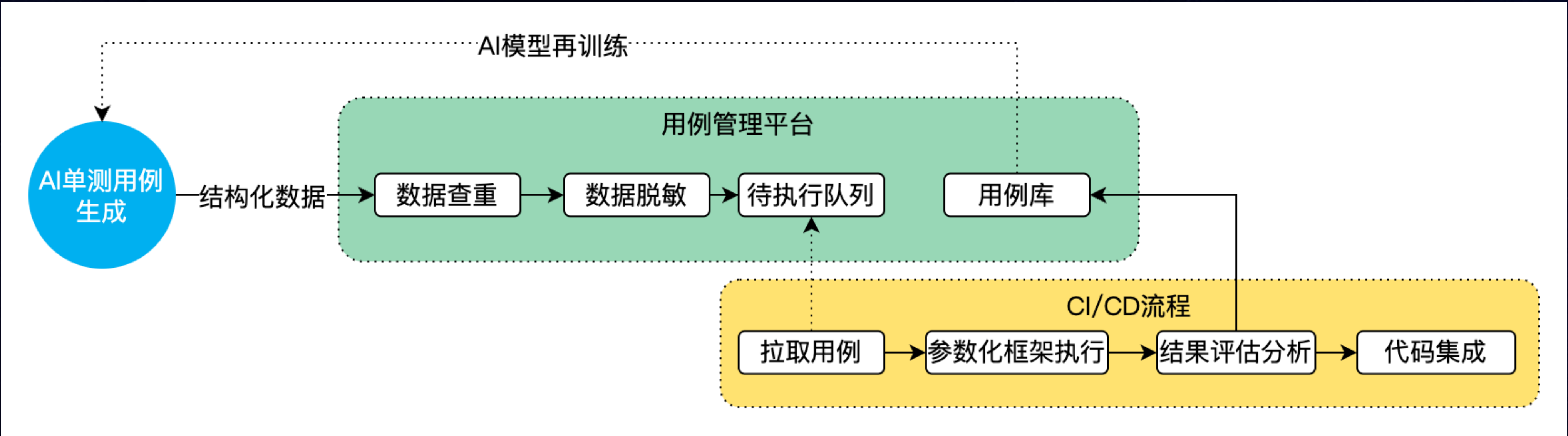
Unittest

负责处理单个函数的异常逻辑分支，保障函数内部逻辑自治。

AndroidTest

负责各模块流程畅通，保障函数/应用之间扭转稳定。

用例数量爆炸式增长



```
1 | @ParameterizedTest
2 | @ValueSource(ints = { 1, 2, 3 })
3 | void testWithValueSource(int argument) {
4 |     assertTrue(argument > 0 && argument < 4);
5 | }
```

参数化测试用例管理

id	suite	page	step	song	p	ver	p1	p2	p3	exp	act	status
623870	7		0	473391799	2	3.2.0	Ht	8.46	8.22			
623902	7		0	230740550	4	3.2.0	SC	0.78	0.46			
623803	7		0	125400077	1	3.2.0	ST	0.42	0.32			
620272	7		0	473391799	2	3.1.7	Ht	8.47	8.22			release
620338	7		0	482090406	6	3.1.7	HI	71.29	71.09			release
619667	7		0	482090406	6	3.2.0	HI	71.28999999	71.09			
620611	7		0	482090406	6	3.2.0	HI	71.28	71.09			release
620437	7		0	358869776	10	3.1.7	臻	49.7	49.37			release
620660	7		0	358869776	10	3.2.0	臻	49.7	49.37			release
620396	7		0	379367725	8	3.1.7	臻	35	34.87			release
620627	7		0	379367725	8	3.2.0	臻	35	34.87			release
619712	7		0	379367725	8	3.2.0	臻	35	34.870000000000005			
619643	7		0	316688109	3	3.2.0	SC	23.24000000	22.950000000000003			
620573	7		0	316688109	3	3.2.0	SC	23.23	22.95			release
620293	7		0	316688109	3	3.1.7	SC	23.21	22.95			release
620370	7		0	427245919	7	3.1.7	臻	2.62	2.43			release
619673	7		0	427245919	7	3.2.0	臻	2.599999999	2.4300000000000007			
620614	7		0	427245919	7	3.2.0	臻	2.58	2.43			release
619730	7		0	4835784	9	3.2.0	臻	189.96	189.45999999999998			
620647	7		0	4835784	9	3.2.0	臻	189.86	189.37			release
620404	7		0	4835784	9	3.1.7	臻	189.85	189.37			release
619741	7		0	358869776	10	3.2.0	臻	154.58999999	154.03999999999996			
620599	7		0	475639296	5	3.2.0	D4	12.4	12.26			release
620331	7		0	475639296	5	3.1.7	D4	12.39	12.26			release
619666	7		0	475639296	5	3.2.0	D4	12.39	12.259999999999998			
620698	7		0	230740550	4	3.1.7	SC	1.24	0.47			release
620298	7		0	230740550	4	3.1.7	SC	0.79	0.47			release
620266	7		0	125400077	1	3.1.7	ST	0.41	0.31			release

PART 04

效果展示

用例覆盖：单元测试用例数增长近6倍

单元测试用例数从111条增长至728条，净增长617条。

——节约人力成本，快速覆盖

效能提升：自动化交付提升占比78.57%

单元测试流水线已执行379次，已有33个版本直接通过自动化交付。

——高效&高质量交付

测试覆盖：代码覆盖率提升20%+

覆盖率代码覆盖率从47%提升至68%；分支覆盖率从29%提升至51%；

——测试场景覆盖更全面

问题发现：提单共计39例

均为单元测试有效提单。

——减少人为错误，确保质量稳定

PART 05

总结与展望

01

模型训练

基于内部代码库和单元测试用例数据，训练大模型进一步提升大模型在单元测试的代码生产能力。提升编译通过率。

02

AI自动归类

AI辅助定位单元测试失败原因，并给出解决方案，减少人工打标成本。同时可基于这点用于后续的模式训练提升。

03

AI自动缺陷修复

在已知代码片段，测试用例的前提下，推测单测发现的缺陷可能的原因，从而帮助开发完成缺陷定位并提供修复建议。

01 单元测试



分析单元测试的优劣势，
结合AI后可以产生1+1>2
的效果。

通过介绍实战项目，了解
如何上手步骤，以及实际
落地后的可能遇到的问题和
解决方案。

03 项目实战



AI工程建设 02

通过合理的架构设计，实
现用例精准生成。

了解用例管理策略以及用
例效果评估可以用到的数
据指标。



落地效果 04

今天讲了什么

AI不是替代测试工程师，而是将人力从重复劳动解放到更高阶的质量设计！

谢谢
THANKS